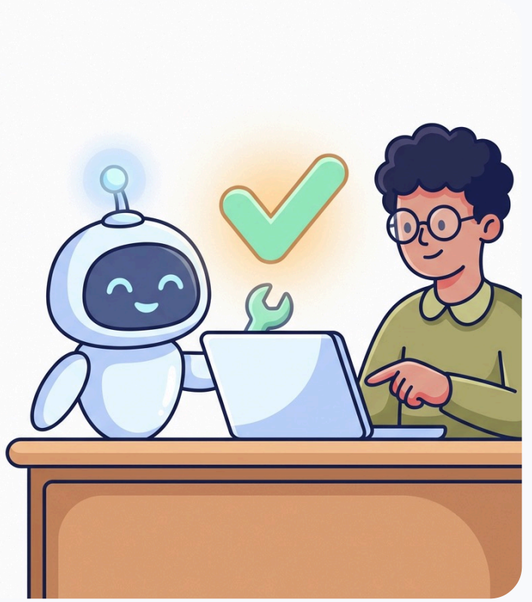
Лекция · агентская разработка

Как бежать быстро и не сломаться



— Николай Иготти

Маршрут

Четыре остановки

1 Что такое агент

цикл, а не автодополнение

2 Три столпа

инструменты, принципы, процессы

3 Где цикл ломается

четыре режима отказа

4 Что сделать

три действия в свой проект

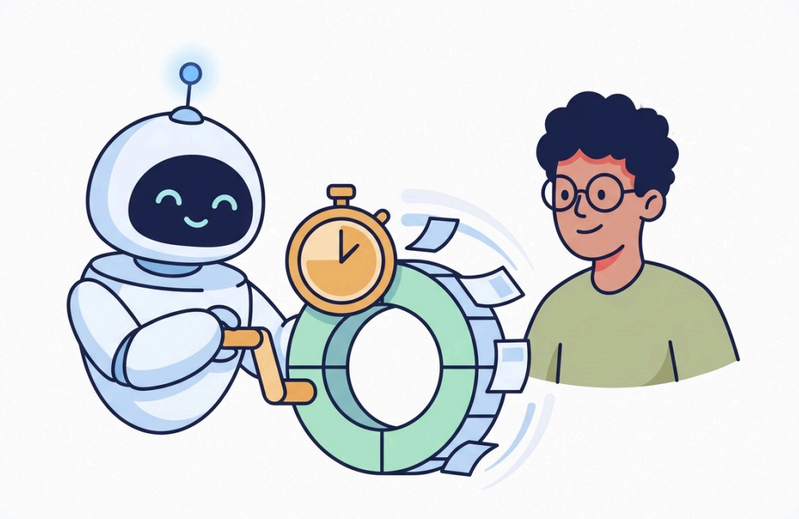
Одна мысль на выходе

Скорость даёт цикл, надёжность — проверка. Всё остальное — способы ускорить одно, не испортив другое.

Что это такое

Агент — это цикл, не автодополнение

- Пишет: сотни строк кода за секунды
- Запускает: компилирует и выполняет сам
- Смотрит: видит результат, а не только код
- Повторяет: правит и запускает снова



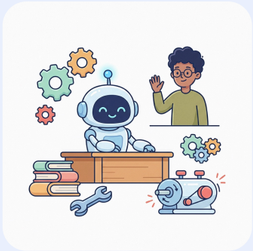
Аналогия

Термостат, а не нагреватель: измеряет, сравнивает с целью и подкручивает — пока не станет ровно.

Организующая рамка

Три составляющих быстрой разработки

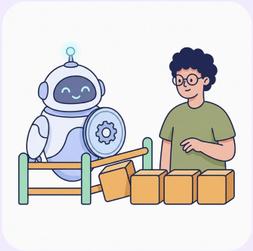
Инструменты



язык, компилятор, среда

дают скорость

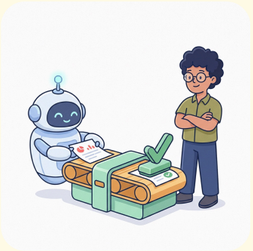
Принципы



стандарты и соглашения

дают стабильность

Процессы



документация, знания

дают темп

Все три нужны одновременно

Без любого из трёх цикл станет медленным, опасным или хаотичным.

Почему это проблема

Три трудности, которые создаёт агент

Дорогая проверка

проверка — второй билет: стоит столько же, сколько генерация

поэтому важна частота циклов, а не размер шага

Вероятностные ошибки

каждая строка — монетка: может быть неверной

надёжность даёт не модель, а слои проверки

Забывание

контекст — оперативка без диска

знания надо выносить в файлы и заметки

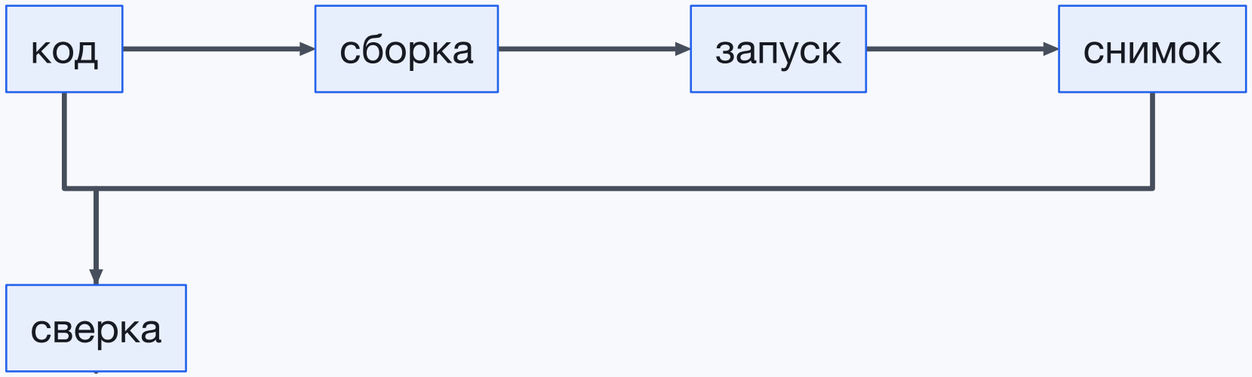
Проверка не бесплатна

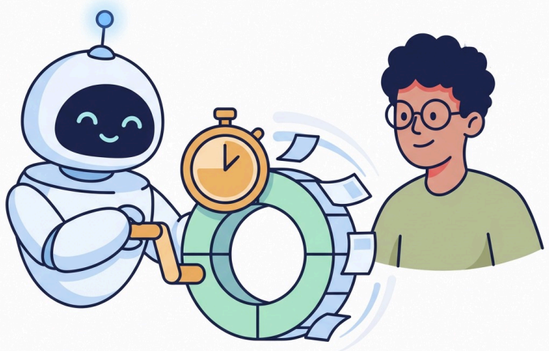
Стоимость работы ≈ число оборотов × (генерация + проверка). Дешёвая проверка делает ошибку дешёвой.

Сердце агентной разработки

Главный цикл: секунды, не минуты

И снова по кругу: один оборот цикла и есть единица работы агента





Что оптимизируем

Не шаг, а оборот: время от правки до сигнала.

Цикл по цифрам

Полный круг за секунды

Порядок величин на ноутбуке: важна не точность, а соотношение

Этап	Время	Что ограничивает
Генерация кода	1–5 с	инференс модели
Компиляция	0,5–2 с	проверка типов
Запуск	0,2 с	среда выполнения
Проверка	0,5 с	рендер и снимок
Полный цикл	≈2–8 с	—

Вопрос аудитории

Что труднее: написать код или понять задачу?

A Написать код

быстро выдать рабочую версию

модель видела миллионы примеров

ошибку видно сразу: код не компилируется

B Понять, что именно нужно

уточнить требования и границы

решить, что считается правильным

ошибку не видно: код работает, но не то

Ответ

Понять задачу — дороже, чем написать код

Код виден

компилятор скажет, что не так

ошибка приходит быстро и на месте

Задача невидима

код работает, но не то

границы и критерии готовности задаёт человек

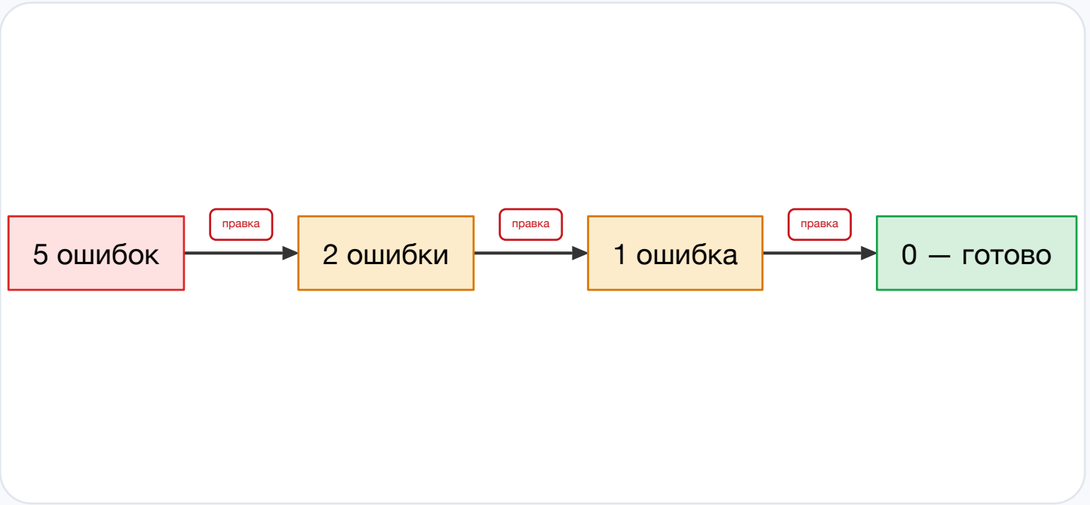
Аналогия

> Код — это черновик; требования — это ТЗ. Проверять надо ТЗ, а не скорость письма.

Как это работает

Сходимость вместо идеального шага

- Каждый шаг неидеален — это нормально
- Цель — последовательность, которая сходится
- Конец — когда сигнал перестал меняться
- Управляем циклом: сигнал → правка → сигнал



Аналогия

Автофокус: линза крутится, пока картинка не станет резкой. Один идеальный поворот не нужен.

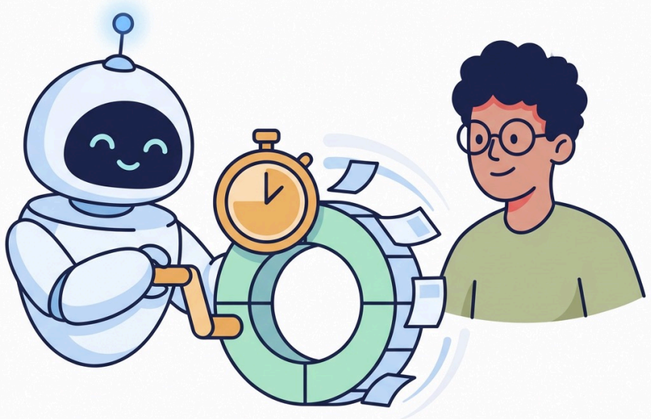
Рычаги скорости

Короче оборот

- Сборка за секунды, а не минуты
- Тесты дают сигнал сразу

Меньше оборотов

- Типы ловят ошибку до запуска
- Отладчик даёт факты, а не догадки
- Маленький шаг: ошибка локальна



Аналогия

Сердцебиение: важнее частота ударов, чем сила одного.

Обратная сторона

Когда процесс не сходится

	Расходимость	Блуждание	Хаос	Ложная сходимость
Признак	Ошибка копится незаметно	Каждая итерация оптимизирует своё	Ошибка не локализована	Тест проходит через раз
Лечить	Короткий автоматический сигнал	Одна цель и критерий готовности	Одна правка — один запуск	Тест должен быть детерминированным

Это не про модель

Все четыре лечатся процессом, а не более сильной моделью.



1

Инструменты

язык · компилятор · среда · отладчик · анализатор · библиотеки

Столп 1 · Инструменты

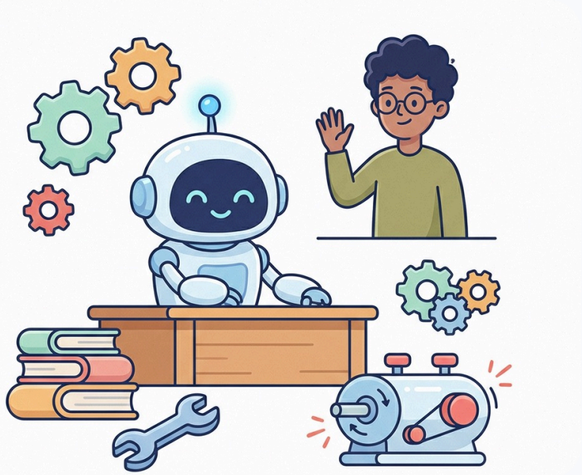
Что ускоряет агента

Язык
статическая типизация

Сборка
секунды, не минуты

Наблюдаемость
отладчик и анализатор

Библиотеки
UI, сеть, хранение



Инструменты — про скорость сигнала

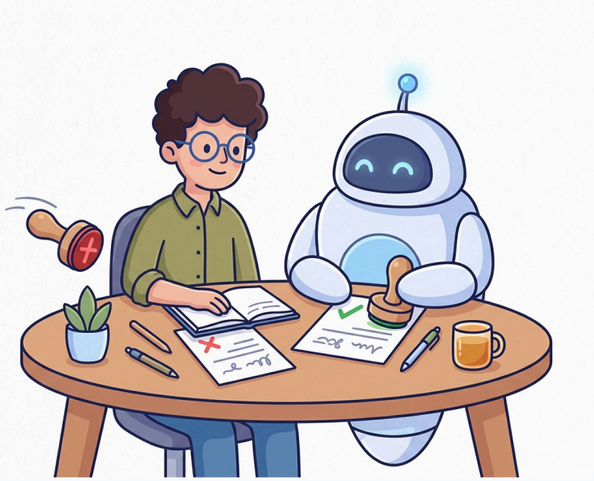
Не «модель умнее», а «ошибка видна быстрее». Хороший инструмент сокращает оборот.

Инструмент №1

Ошибка видна до запуска

ARKTS

```
const n: number = "текст"   // ✗
const n: number = 42        // ✓
```



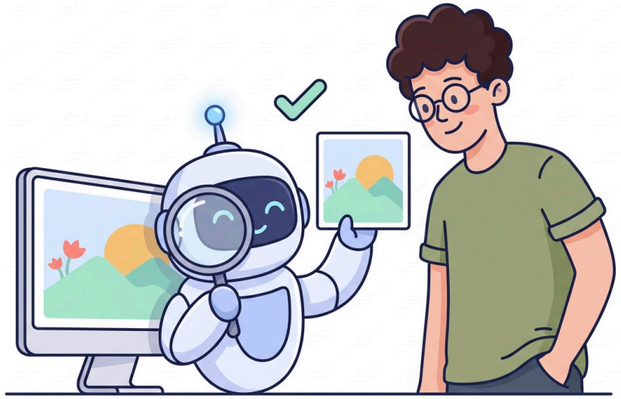
- Агенту не нужно запускать код, чтобы узнать о неверном типе
- Ошибка приходит за секунды и там, где она сделана

> Проверка орфографии — до отправки, не после.

Инструмент №2

«Глаза и руки» агента

- Видит отрисованный результат, а не только код
- Ходит по дереву интерфейса и кликает
- Сравнивает с ожидаемым состоянием



Аналогия

Смотреть на код — как читать меню на незнакомом языке. Снимок и дерево — это переводчик.



2

Принципы

стандарты · соглашения · защитные перила

Столп 2 · Принципы

Защитные перила

Никогда: репозиторий

публикация в публичный реестр

перезапись истории

необратимое удаление файлов

Никогда: доступы

запуск чужого кода с правами

правка секретов и ключей

«добавить всё» одной командой

Только по просьбе

публикация артефактов

массовая очистка репозитория

изменение конфигурации CI

Перила делают ошибку обратимой

Они не замедляют движение — они не дают уехать туда, откуда нет возврата.

Принципы · Самопроверка

Кто что проверяет перед сдачей

Проверка	Агент	Человек
Сборка и типы	✓	
Тесты и граничные случаи	✓	✓
Дифф и посторонние файлы	✓	✓
Требования и критерии готовности		✓
Секреты, доступы и последствия		✓

Разделение труда

Машина проверяет видимое; человек — смысл, границы и последствия.

Итог принципов

Эшелонированная оборона

Чем раньше слой ловит ошибку — тем дешевле исправление

4 · Среда выполнения
дорого, ловит мало

3 · Сверка отрисовки
видно, что получилось

2 · Валидация
данные и границы

1 · Типы
дёшево, ловит больше всего

Аналогия: досмотр в аэропорту

Каждая проверка ловит то, что пропустила предыдущая; ошибка дорожает вглубь.



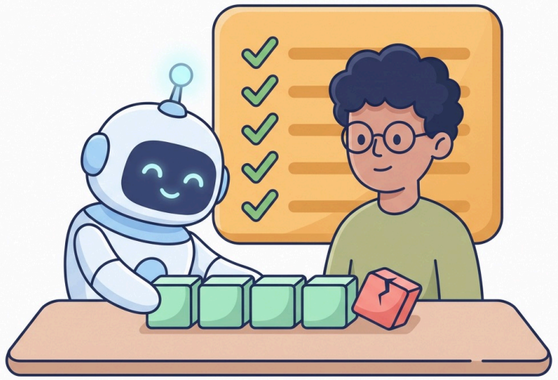
3

Процессы

документация · ревью · CI · тесты · база знаний

Тесты и CI: предсказуемый агент

- Тесты пишутся вместе с функцией, а не после
- Граничные случаи — там, где живут ошибки
- Пирамида: много быстрых юнит-тестов, меньше интеграционных, единицы end-to-end
- Тест обязан падать до правки и проходить после
- CI: установка, сборка и тесты на каждом коммите



Аналогия

Тест — датчик дыма, CI — ночной сторож: срабатывают без вас и не устают к третьему часу ночи.

Зелёный сигнал должен что-то значить

✖ Плохо: тест без проверки

ARKTS

```
test("сложение", () => {
  print("ок")
})
```

✔ Хорошо: тест утверждает

ARKTS

```
test("сложение", () => {
  assert(sum(2, 2) == 4)
})
```

Аналогия

Тест без assert — датчик, отключённый от провода.

Процессы · CI

Красный пайплайн — это задача

1 Упало

лог, шаг, первая ошибка

2 Понять

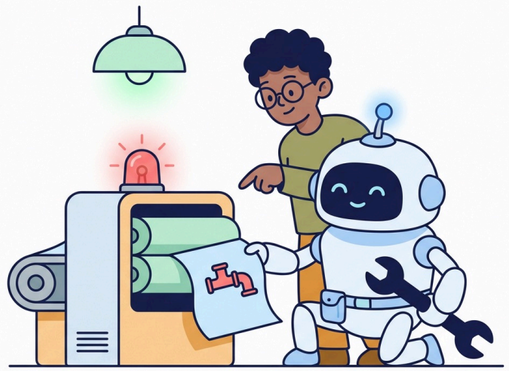
воспроизвести локально

3 Починить

правкой в репозитории

4 Перезапустить

зелёный не случаен?

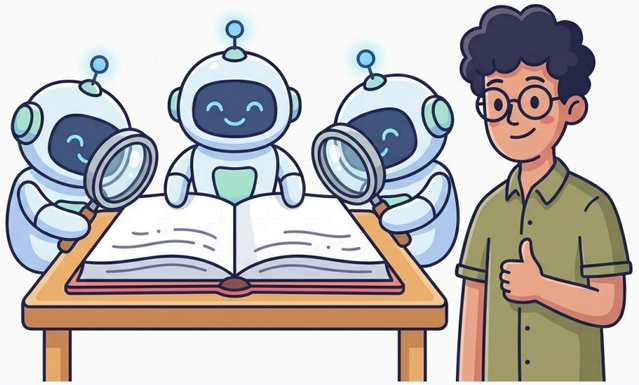


Не «перезапустить», а прочитать

Красный пайплайн — письмо от ночного сторожа. Чинить причину, а не перезапускать до зелёного.

Ревью: независимые взгляды

- Четыре роли: архитектура, безопасность, скорость, соглашения
- У каждого свой чек-лист рядом с определением
- Ревью ловит то, чего не видит CI: замысел, границы, последствия
- Одобряет человек: иначе проверка сертифицирует сама себя



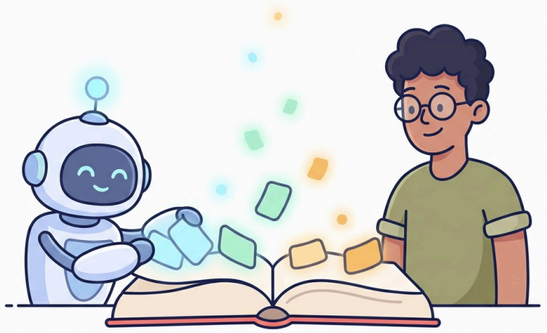
Аналогия

Второй пилот в кабине: не потому что первый хуже, а потому что один взгляд устаёт и привыкает.

Процессы · Знание

База знаний и память агента

- «Почему» и «как связано» — рядом с кодом
- Заметки живут между сессиями агента
- Правила и заметки — вход агента, а не отчёт для людей
- Устаревшая заметка хуже отсутствующей
- Обновляется вместе с изменением, а не после



Аналогия

Контекст — оперативная память, заметки — диск. Что не записано, теряется при первой же перезагрузке.

Пример · Как это выглядит

Один конкретный стек как иллюстрация

Что даёт агенту дружелюбный инструментарий

- Компиляция и запуск — секунды, не минуты
- Статические типы ловят ошибку до запуска
- Тесты без экрана и «глаза» на интерфейс
- Внешняя память: правила и заметки рядом с кодом

Пример стека

gitcode.com/rri_opensource/arkui3

Берите свойства, а не бренд

Те же свойства собираются из других инструментов.

Что взять с собой

Чек-лист агентной разработки

Что не проверяется машиной — того не существует

Правила

файл, который агент читает до работы

Короткий цикл

компиляция и проверка за секунды

Ранние ошибки

типы и валидация до запуска

«Глаза»

снимок и дерево вместо слепого кода

Тесты с кодом

и запуск на каждом коммите

База знаний

«почему» и «как связано»

Что сделать на этой неделе

Три действия в свой проект

1 Правила

создайте файл, который агент читает до работы

границы, запреты, как проверять

2 Сигнал

добейтесь сборки и тестов за секунды

одна команда — и видно результат

3 Один тест

напишите падающий тест до правки

затем попросите агента его починить

Минимальный цикл

Правила + быстрый сигнал + один тест. Всё остальное — улучшения поверх этого.